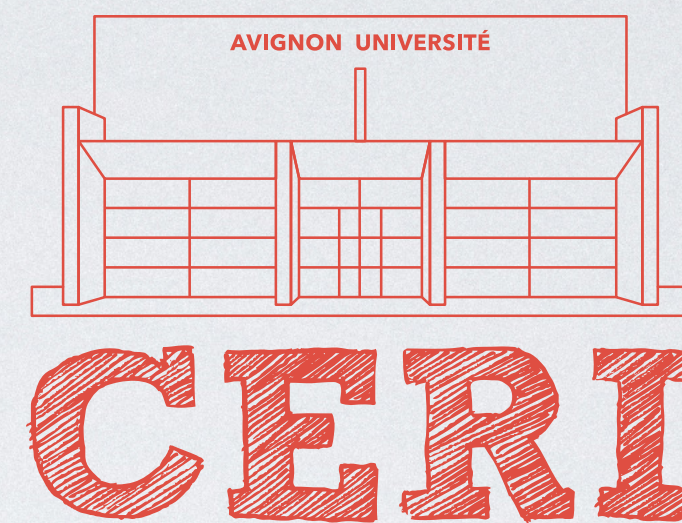




Processus

Introduction aux systèmes d'exploitation

Teva MERLIN

Temps partagé, multitâche

- Années 60 : apparition des systèmes à temps partagé
 - CTSS (1963) → Multics (1969) → Unix (1971)
- Ordinateurs personnels :
 - initialement : fonctionnement monotâche
 - milieu années 80 → multitâche (majoritairement coopératif)
 - fin des années 90 → multitâche préemptif

Processus

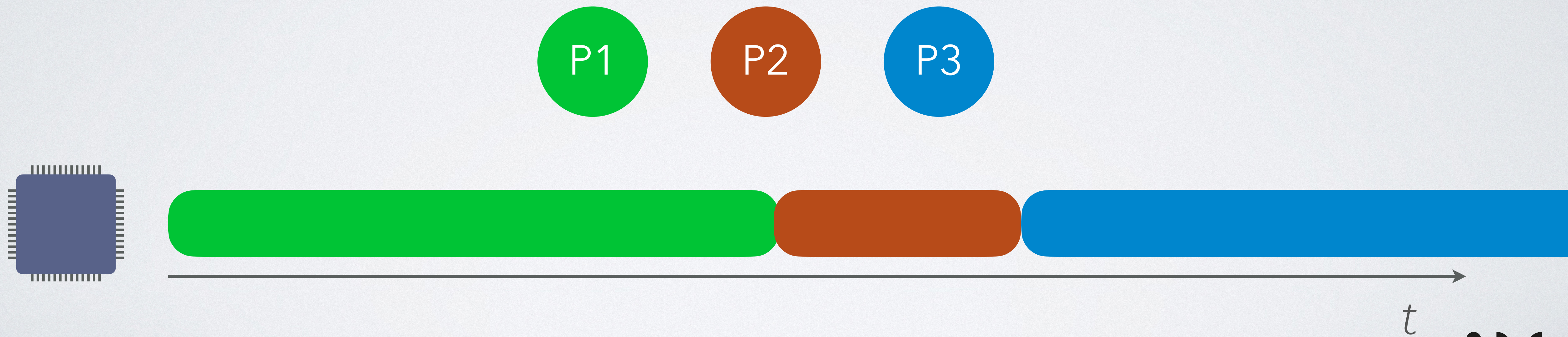
- Tâche à exécuter (application, service de l'OS, etc.)
- Propriétés :
 - Fichier exécutable
 - Propriétaire, droits d'accès
 - Date de lancement, temps d'exécution, priorité
 - Fichiers ouverts
 - Espace mémoire

Lister les processus

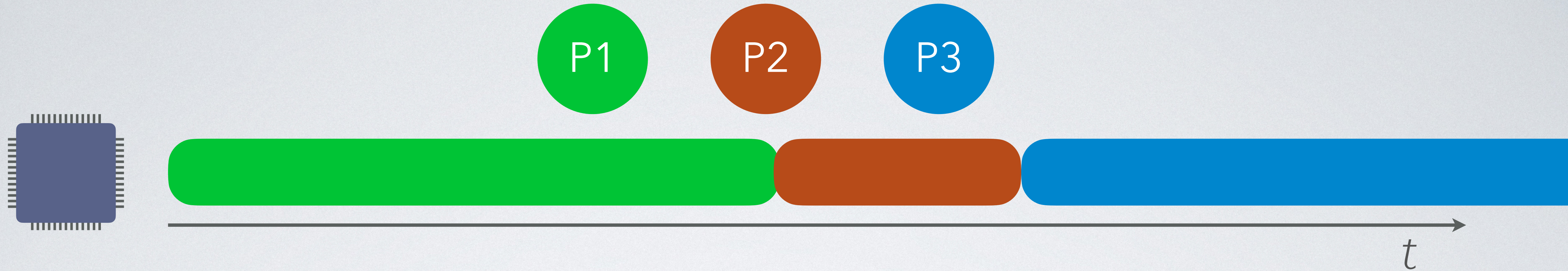
- `ps`
 - Options : `-f`, `-u`, `-x`, `-a` et combinaisons
- `pstree`
- `top`, `htop`
- Gestionnaire de tâches / Moniteur d'activité

1 cœur CPU $\Leftrightarrow$ 1 seul processus actif à chaque instant

Traitement par lot / Système monotâche



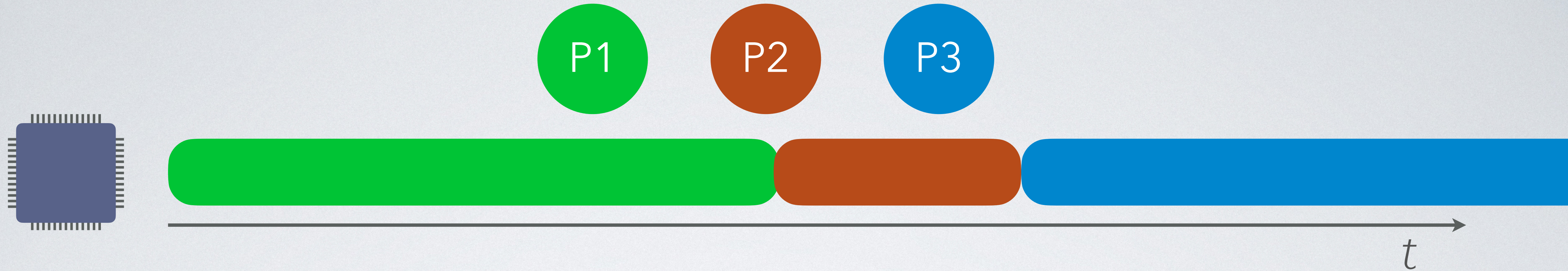
Traitement par lot / Système monotâche



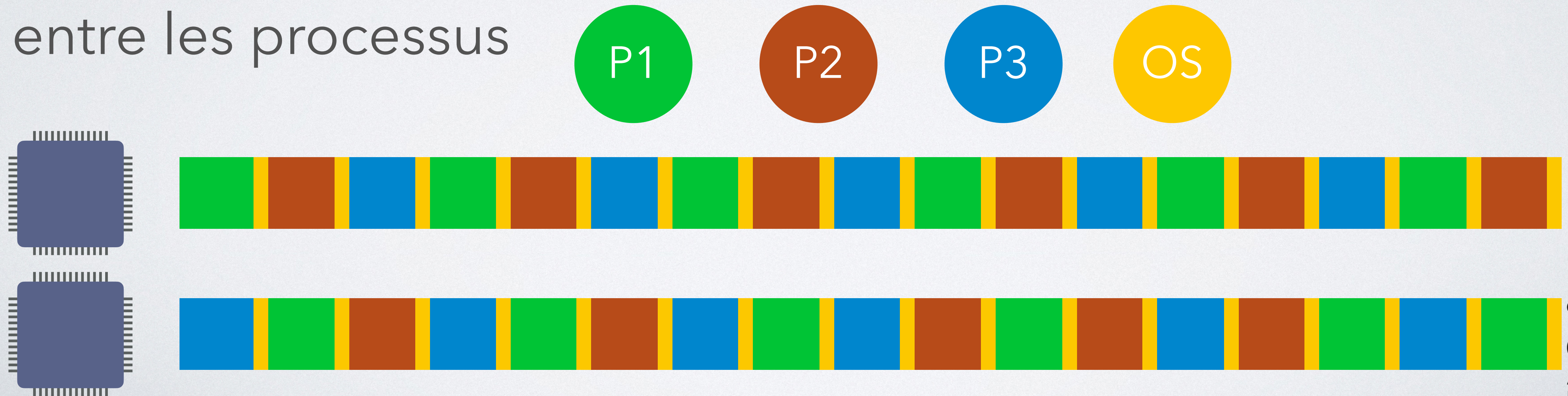
Temps partagé : ordonnanceur pour distribuer le temps CPU entre les processus



Traitement par lot / Système monotâche



Temps partagé : ordonnanceur pour distribuer le temps CPU entre les processus



Ordonnanceur : objectifs

Maximiser

- Efficacité (% de CPU utilisé)
- Rendement
- Équité

Minimiser

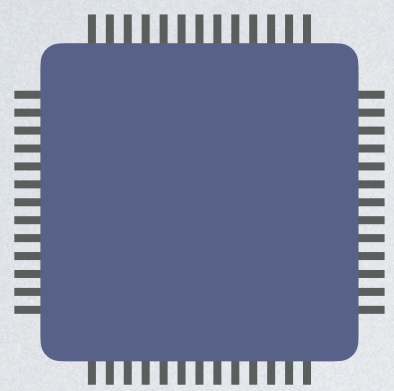
- Temps de réponse
- Temps de traitement

Tourniquet

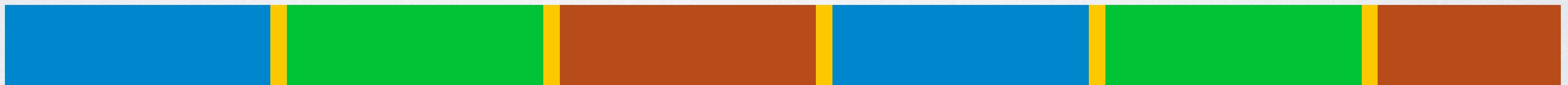
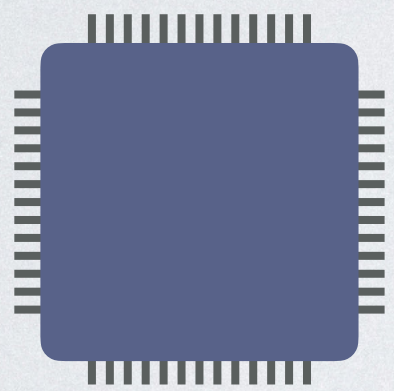
- Ordonnancement circulaire
 - File d'attente de type FIFO
 - 1 quantum de temps pour le processus en tête de file
- ⚠ La taille du quantum est cruciale pour un bon résultat



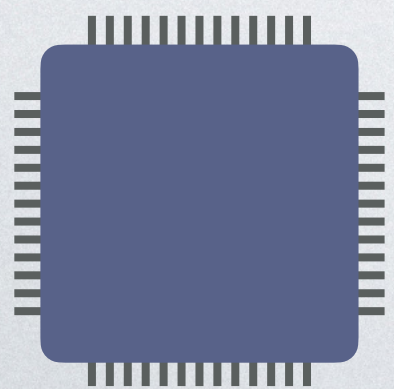
Quantum trop petit $\Rightarrow$ pourcentage élevé de temps consacré à l'ordonnancement



Quantum trop grand $\Rightarrow$ temps de réponse trop faible pour les processus interactifs / temps réel



Compromis à trouver



Temps de changement de processus

- Exécution de l'algorithme d'ordonnancement
 - généralement pas très long
- Changement de contexte
 - Vidage des registres et des caches CPU
 - Coût CPU élevé
 - Multi-CPU / multi-cœurs à prendre en compte

Priorité

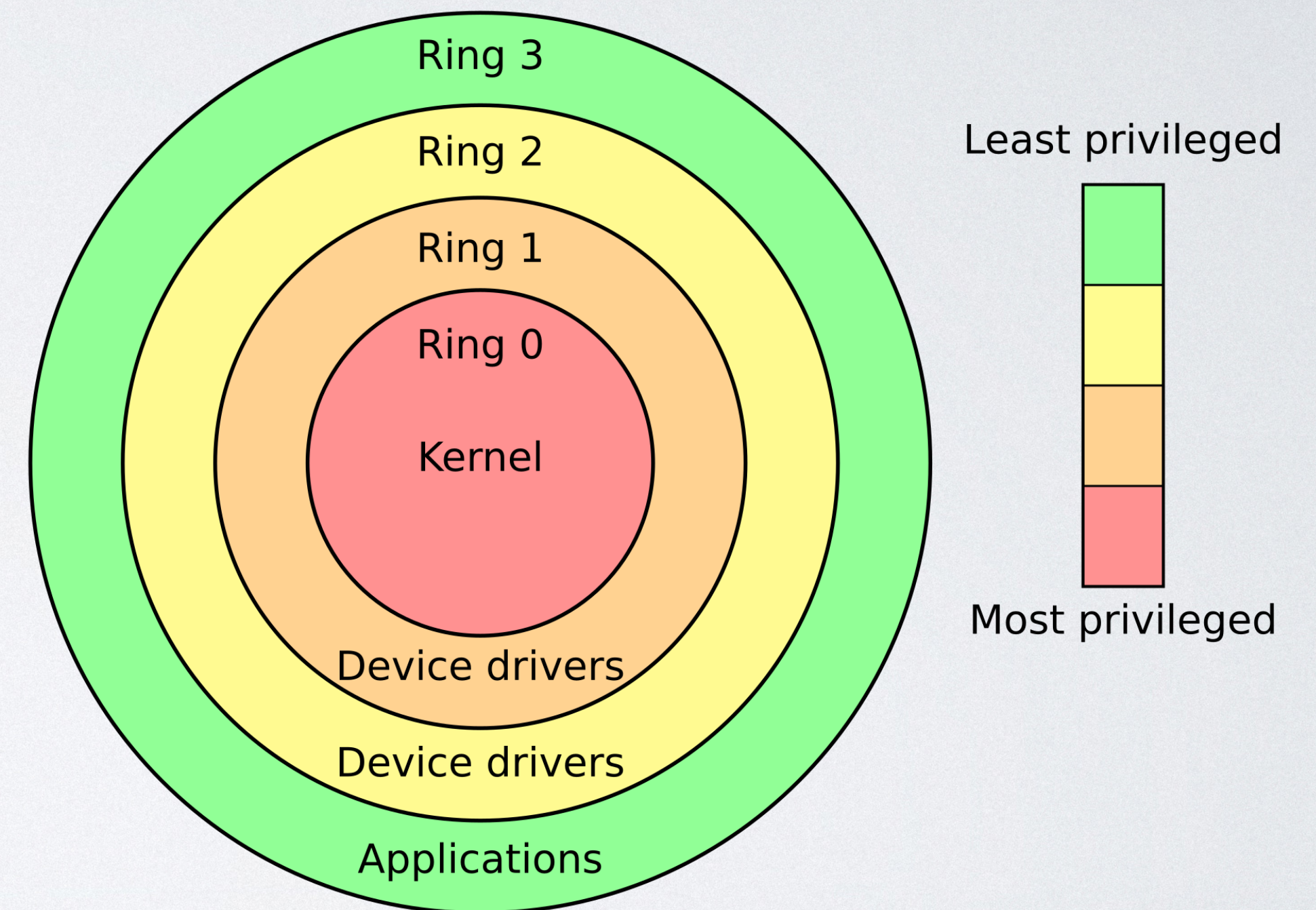
- Ajustement du tourniquet pour prendre en compte la priorité des processus
 - 1 file d'attente par niveau de priorité
- Sur Unix :
 - Priorité de -20 (max) à 20 (min), à 0 par défaut
 - Changement avec `nice`, `renice`

Évolution des contraintes

- Systèmes conçus pour des gros serveurs multi-utilisateurs
- Aujourd'hui, beaucoup de machines :
 - Personnelles, mono-utilisateur
 - Fonctionnant sur batterie $\Rightarrow$ cœurs hétérogènes (big.LITTLE)
 - Avec usage via une interface graphique

Gestion des processus

- Noyau de l'OS, appels système
- Support matériel :
 - Interruptions $\Rightarrow$ préemption
 - Protection : modes CPU / anneaux



Espace mémoire et protection

Mémoire virtuelle paginée

- Espace RAM virtuel pour chaque processus
 - adresse virtuelle $\neq$ adresse physique
- Accès protégé vis-à-vis des autres processus
- Paginé $\Rightarrow$ *swap*
- Gestion matérielle : PMMU (*Paged Memory Management Unit*)

Threads

(« *fils* », « *processus légers* »)

- Dépendent d'un processus
- Partagent l'espace mémoire du processus
- Usages :
 - Traitements de fond
 - Accélération des traitements parallélisables

Évolution des contraintes

- Protection des données vis-à-vis des autres utilisateurs → protection des données de l'utilisateur vis-à-vis de ses propres processus
- Sandbox
- Applications multi-processus, pour faire tourner des parties dans des sandboxes plus restrictives et améliorer la stabilité

Communication inter-processus

(IPC — inter-process communication)

- Multiples techniques proposées par les OS
- Ou par des frameworks
- IPC historique de base sur Unix : signaux
 - envoyés avec `kill`